

Clase 3 : Programación

Introducción a la programación.

Temario

- Programación modular.
 - Creación de módulos
 - Funciones
 - Parámetros
- Recursión
 - Concepto y características
 - Funciones recursivas



Módulos y funciones

Módulos

Son archivos de texto cuya extensión es: .py, .pyc o .pyw

Contienen instrucciones Python.

Se pueden definir funciones dentro del módulo.

Se puede ejecutar cuantas veces se quiera.

Se los puede importar y usar desde otros módulos.

Sentencia import

Con **import** se importan módulos. De esta forma, podemos acceder a su contenido.

Forma general

```
import nombre_modulo
```

¿Qué significa “importar” un módulo?

Se tiene “acceso” a los datos y las funciones definidas dentro del módulo.

Sentencia import

Para invocar una función definida dentro del módulo:

```
nombre_modulo.nombre_funcion
```

También se puede usar:

```
from nombre_modulo import lista de nombres de funciones
```

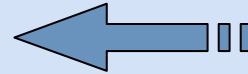
En este caso no debe llamarse a la función con el nombre del módulo.

Definiendo Funciones

```
def nombreFuncion(parametros):
```

```
    sentencias
```

```
    return <expresion>
```



El cuerpo de la función **debe** estar indentado!

Ejemplo sencillo:

```
def cuadrado(x):
```

```
    return x*x
```

x es el parámetro de la función

¿Cómo lo usamos?

```
print(cuadrado(3))
```

```
a=2+cuadrado(9)
```

Pasaje de parámetros

```
def nombreFuncion(parámetros):
```

```
    sentencias
```

```
    return <expresion>
```

Los parámetros se pueden pasar por copia o referencia. No se identifica con ninguna palabra clave el pasaje.

Pasaje de Parámetros

- Los parámetros deben pasarse respetando cantidad y posición.

Pero Python **permite omitir alguno**, definir en la llamada la relación entre el actual y el formal, o usar una cantidad variable de parámetros.

Pasaje de Parámetros

```
def suma(x,y):  
    return x + y
```

¿Cómo lo
usamos?

```
print(suma(3,4) )
```

□ Imprime 7

```
def pot(x,y=2):  
    return x **y
```

¿Cómo lo
usamos?

```
print(pot(3))
```

□ imprime 9

```
def suma(x,y):  
    return x + y
```

¿Cómo lo usamos?

```
print(suma(y=5, x=4) )
```

□ Imprime 9

Variables locales y globales

```
x=12
```

```
a=13
```

```
def mi_funcion(a):
```

```
    x=9
```

```
    y=10
```

Usa la x local

Variables locales enmascaran las globales

Acceso a las variables globales mediante **global**

```
def miFuncion(a):
```

```
    global x
```

```
    x=9
```

```
    a=10
```

Ejercicio 1

- Definir una función que reciba una lista de precios y retorne el precio promedio.

Ejercicio 1

- Definir una función que reciba una lista de precios y retorne el precio promedio.

```
def promedio( lisprecios):  
    suma=0  
    cant= len(lisprecios)  
    for j in range (0, cant, 1):  
        suma=suma + lisprecios[j]  
    prom=suma/cant  
    return prom
```

[200,450,180]



Recursión- Funciones recursivas

Recursión

- Es una técnica que **permite definir un problema en términos de si mismo.**
- La recursión es una técnica de programación muy poderosa, en la cual **una función realiza llamadas a sí misma (se auto invoca)** en pos de resolver un problema.

Recursión

- Razones para su uso:
 - Problemas “casi” irresolubles mediante estructuras iterativas.
 - Problemas de naturaleza recursiva (combinatoria, serie de fibonacci, etc.).
 - Cuando el lenguaje de programación no provee estructuras de control iterativas (prog. Funcional y lógica).

Recursión

- Un algoritmo recursivo presenta 3 características:
 -
- **Tiene al menos un Caso Recursivo:** aquí la función realiza algunas operaciones con las que ***se reduce la complejidad*** del problema y luego realiza ***un llamado a si misma*** (auto invocación).
- **Tiene al menos un Caso Base:** Es un caso especial, cuando el cálculo es tan simple que se puede resolver **directamente** sin necesidad de hacer una llamada recursiva. Corta el proceso repetitivo.
- En cada llamada recursiva se reduce la complejidad del problema.

Recursión - Ejemplo

- Definir una función recursiva que reciba un número natural y devuelva el factorial de dicho número

```
def fact(n):  
    if (n==0):  
        return 1  
    else:  
        return (n * fact(n-1))
```

$n!(n-1)! \dots 1$
 $0!=1$
 $n!=n * (n-1)!$

Recursión - Ejemplo

- Definir una función recursiva que reciba un número natural y devuelva el factorial de dicho número

```
def fact(n):  
    if (n==0):  
        return 1  
    else:  
        return (n * fact(n-1) )
```

$0! = 1$

$n! = n * (n-1)!$

Caso base:
La función finaliza con las
llamadas recursivas.

Recursión - Ejemplo

- Definir una función recursiva que reciba un número natural y devuelva el factorial de dicho número

```
def fact(n):  
    if (n==0):  
        return 1  
    else:  
        return (n * fact(n-1) )
```

$0! = 1$

$n! = n * (n-1)!$

Caso recursivo: se autoinvoca.
Reduce el problema, achica el
valor de n

Recursión - Ejemplo

- Definir una función recursiva que reciba un número natural y devuelva el factorial de dicho número

- `def fact(n):`

- `if (n==0):`

- `return 1`

$0! = 1$

- `else:`

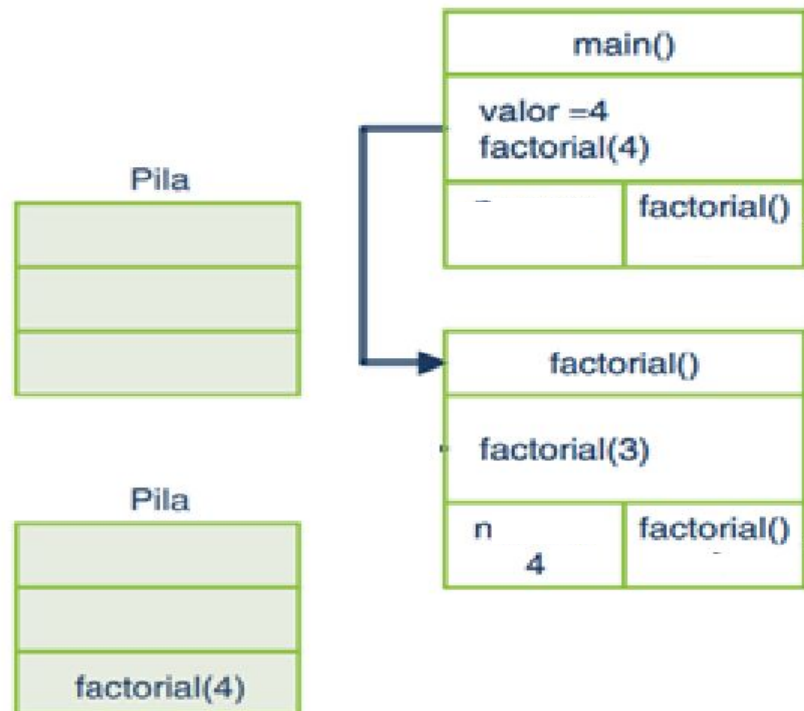
- `return (n * fact(n-1) )`

$n! = n * (n-1)!$

Desde el Main invocamos por ejemplo: `res=fact(4)`

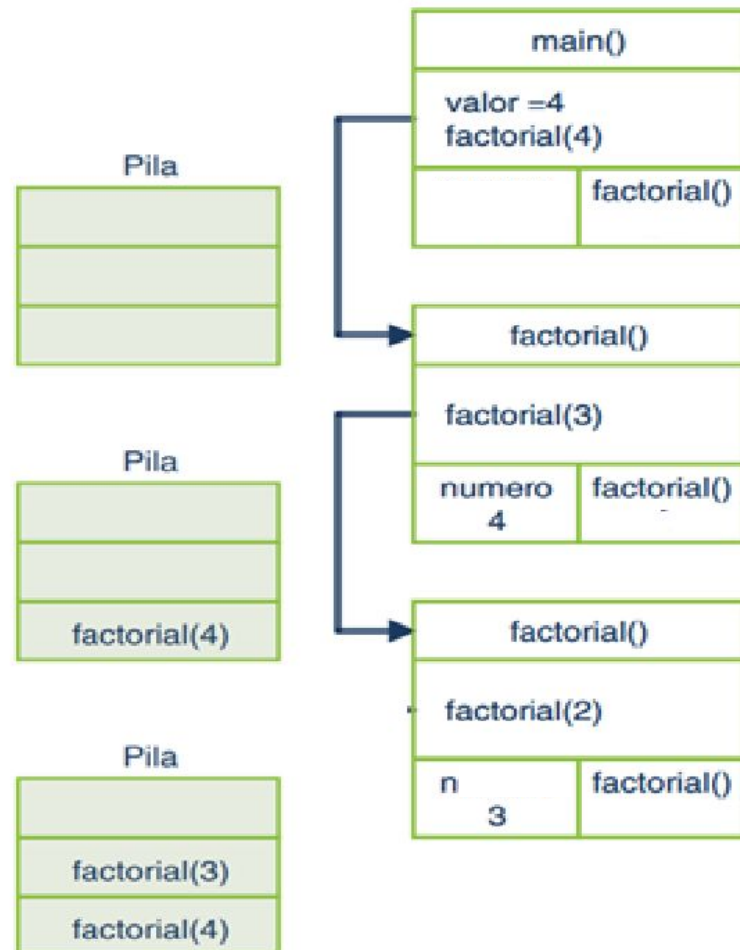
Pila de ejecución

```
• def fact(n):  
    if (n==0):  
        return 1  
    else:  
        return (n * fact(n-1) )
```



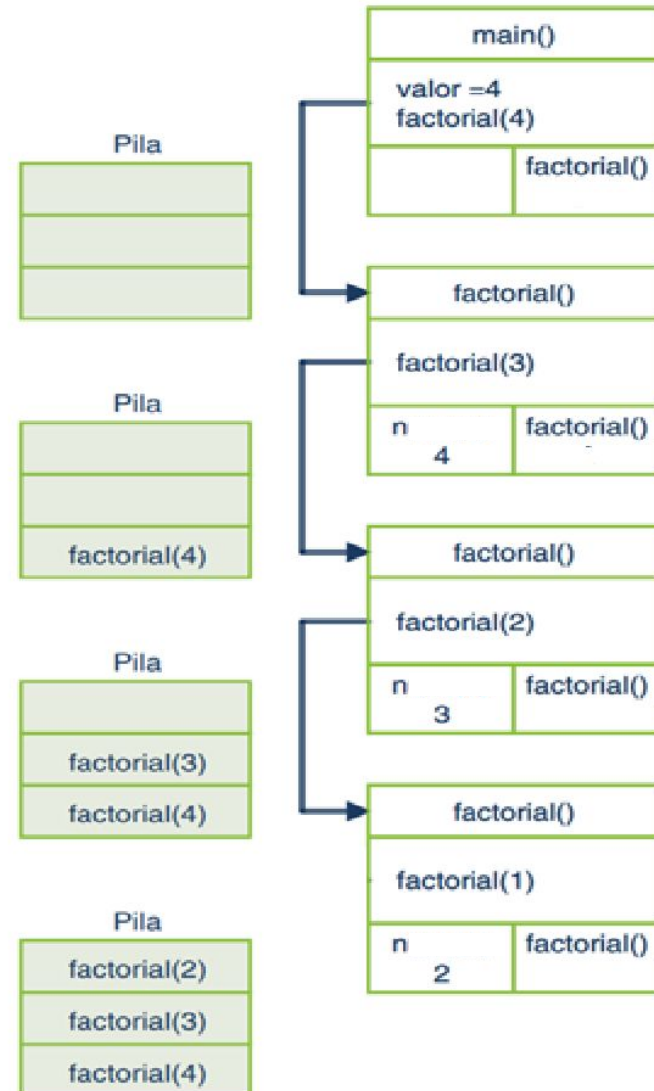
Pila de ejecución

```
def fact(n):  
    if (n==0):  
        return 1  
    else:  
        return (n * fact(n-1) )
```



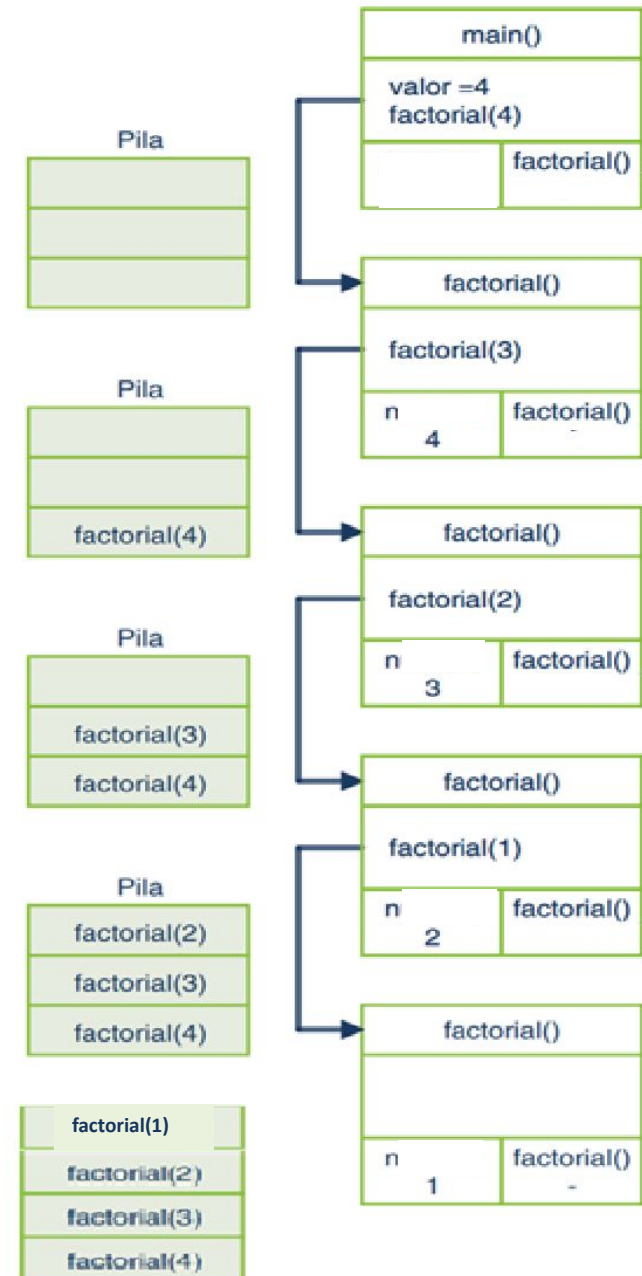
Pila de ejecución

```
• def fact(n):  
    if (n==0):  
        return 1  
    else:  
        return (n * fact(n-1) )
```



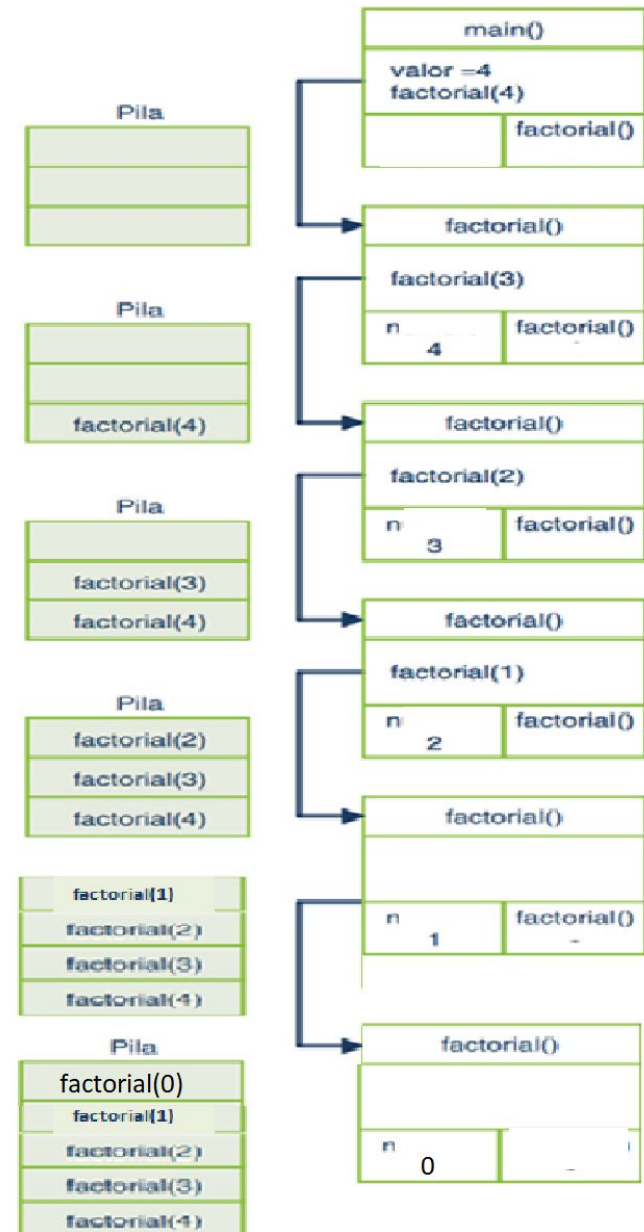
Pila de ejecución

```
def fact(n):  
    if (n==0):  
        return 1  
    else:  
        return (n * fact(n-1) )
```



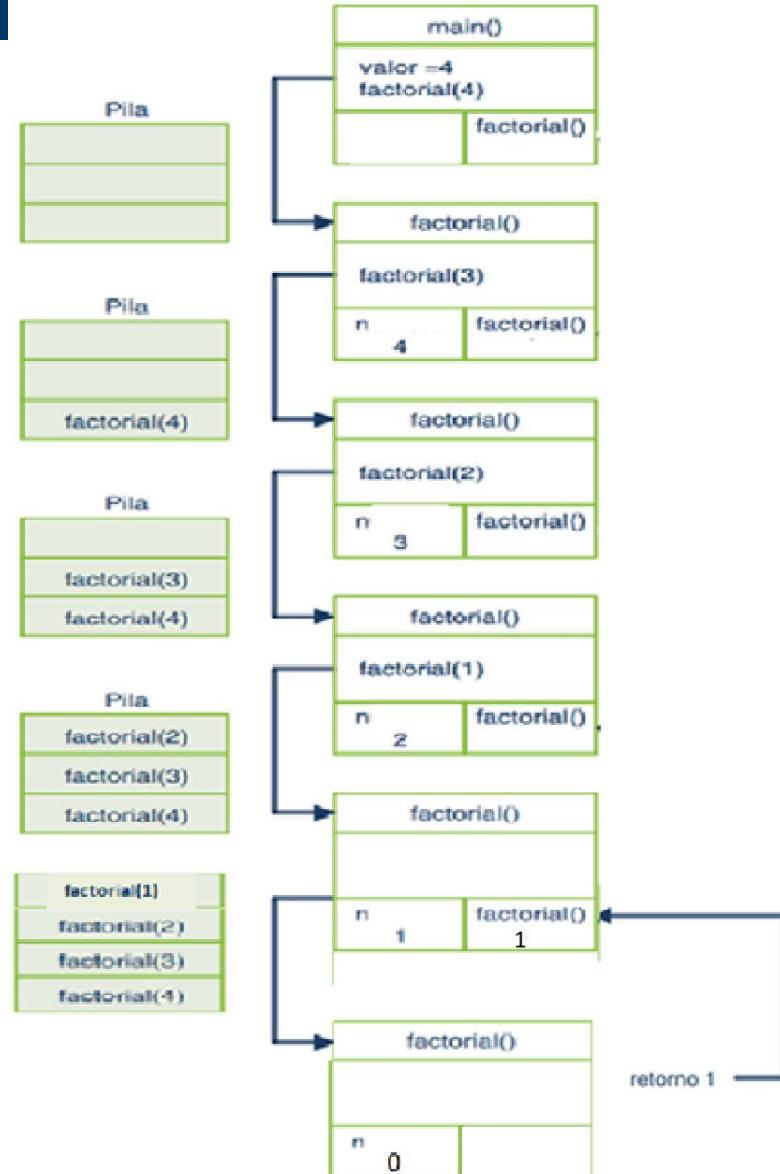
Pila de ejecución

```
def fact(n):  
    if (n==0):  
        return 1  
    else:  
        return (n * fact(n-1) )
```



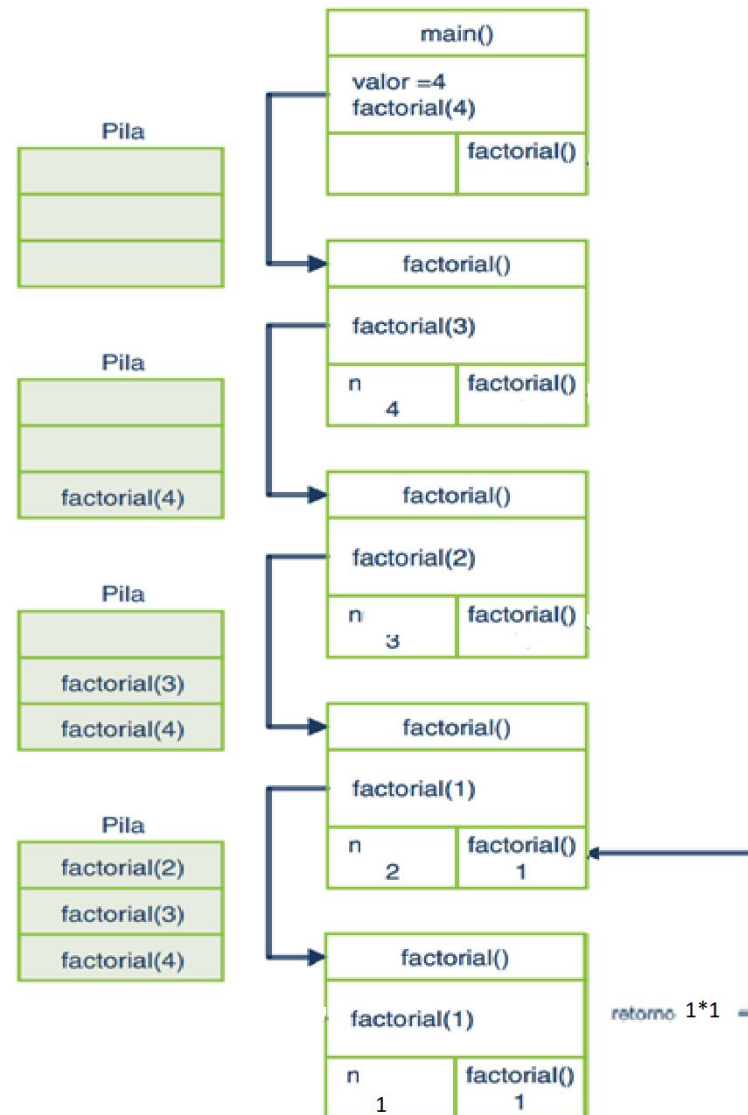
Pila de ejecución

```
def fact(n):  
    if (n==0):  
        return 1  
    else:  
        return (n * fact(n-1) )
```



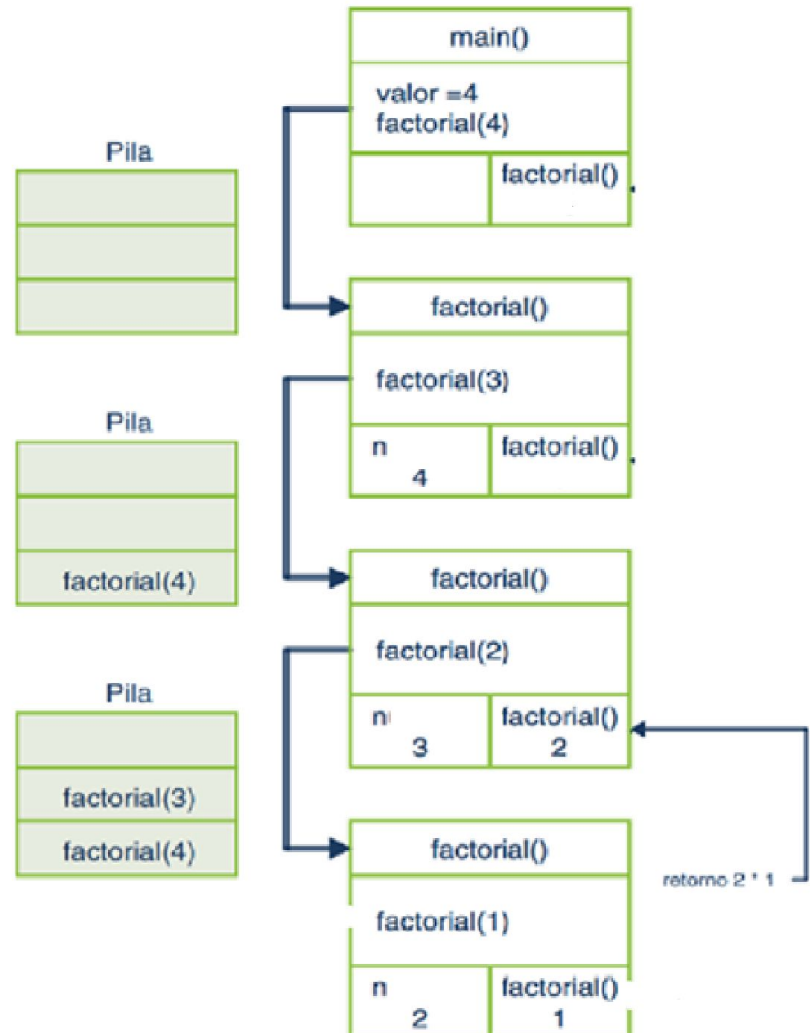
Pila de ejecución

```
def fact(n):  
    if (n==0):  
        return 1  
    else:  
        return (n * fact(n-1) )
```



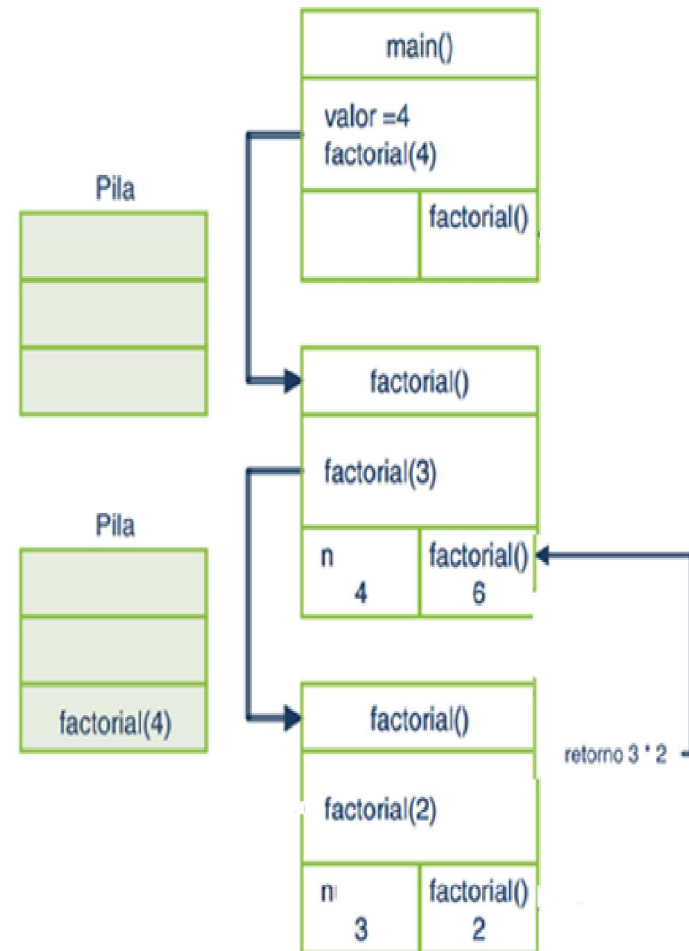
Pila de ejecución

```
def fact(n):  
    if (n==0):  
        return 1  
    else:  
        return (n * fact(n-1) )
```



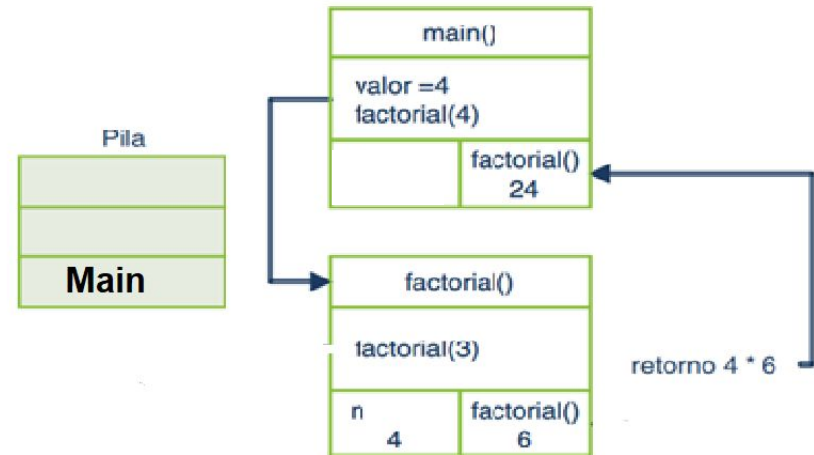
Pila de ejecución

```
def fact(n):  
    if (n==0):  
        return 1  
    else:  
        return (n * fact(n-1) )
```



Pila de ejecución

```
• def fact(n):  
    if (n==0):  
        return 1  
    else:  
        return (n * fact(n-1) )
```



La técnica de diseño recursiva construye el resultado en muchos casos mediante un proceso de **backtracking**, es decir, en el camino de retroceso desde el caso base, por retroalimentación

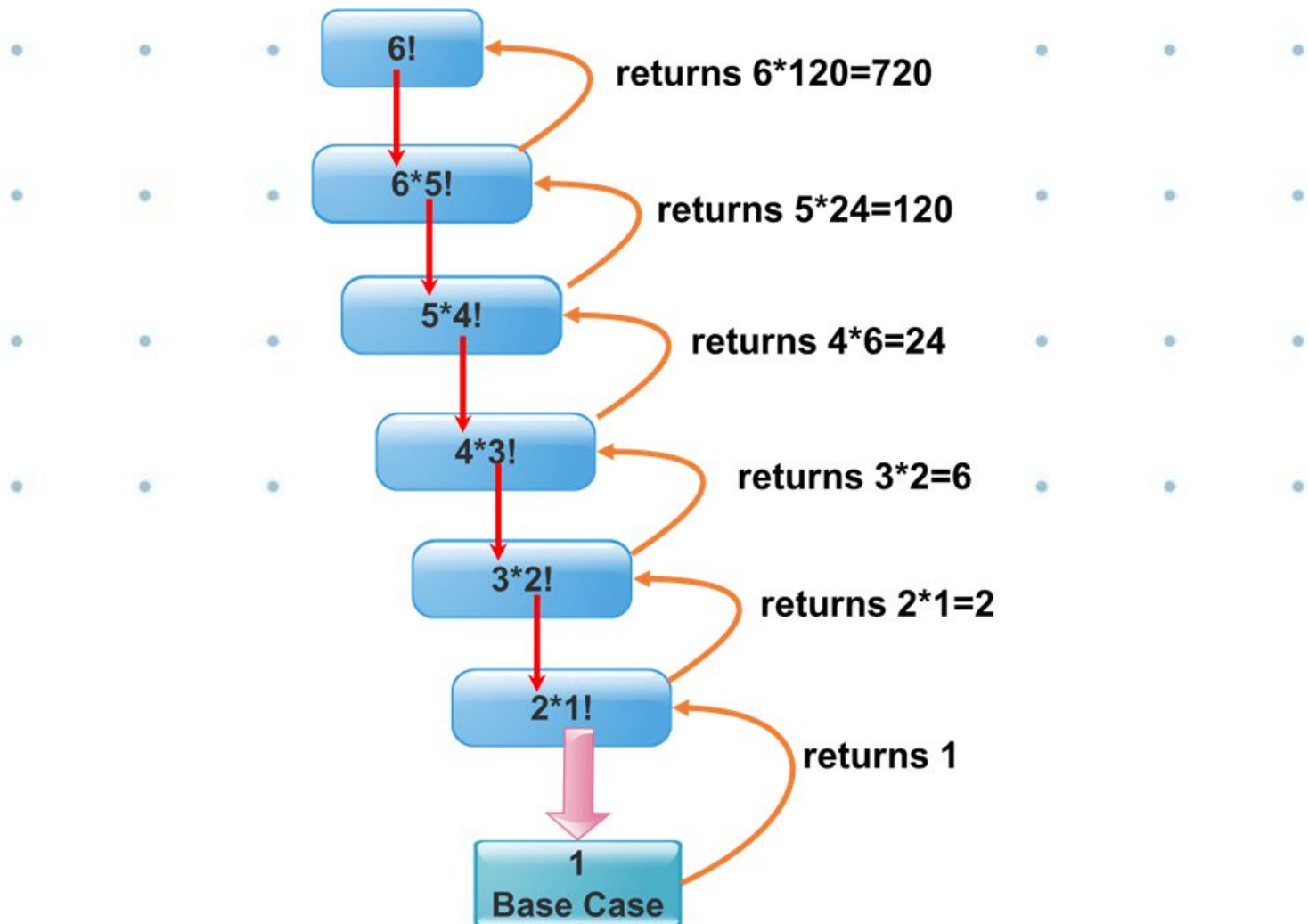
Recursión

```
factorial(6)
6 * factorial(5)
6 * (5 * factorial(4))
6 * (5 * (4 * factorial(3)))
6 * (5 * (4 * (3 * factorial(2))))
6 * (5 * (4 * (3 * (2 * factorial(1)))))
6 * (5 * (4 * (3 * (2 * 1))))
6 * (5 * (4 * (3 * 2)))
6 * (5 * (4 * 6))
6 * (5 * 24)
6 * 120
720
```

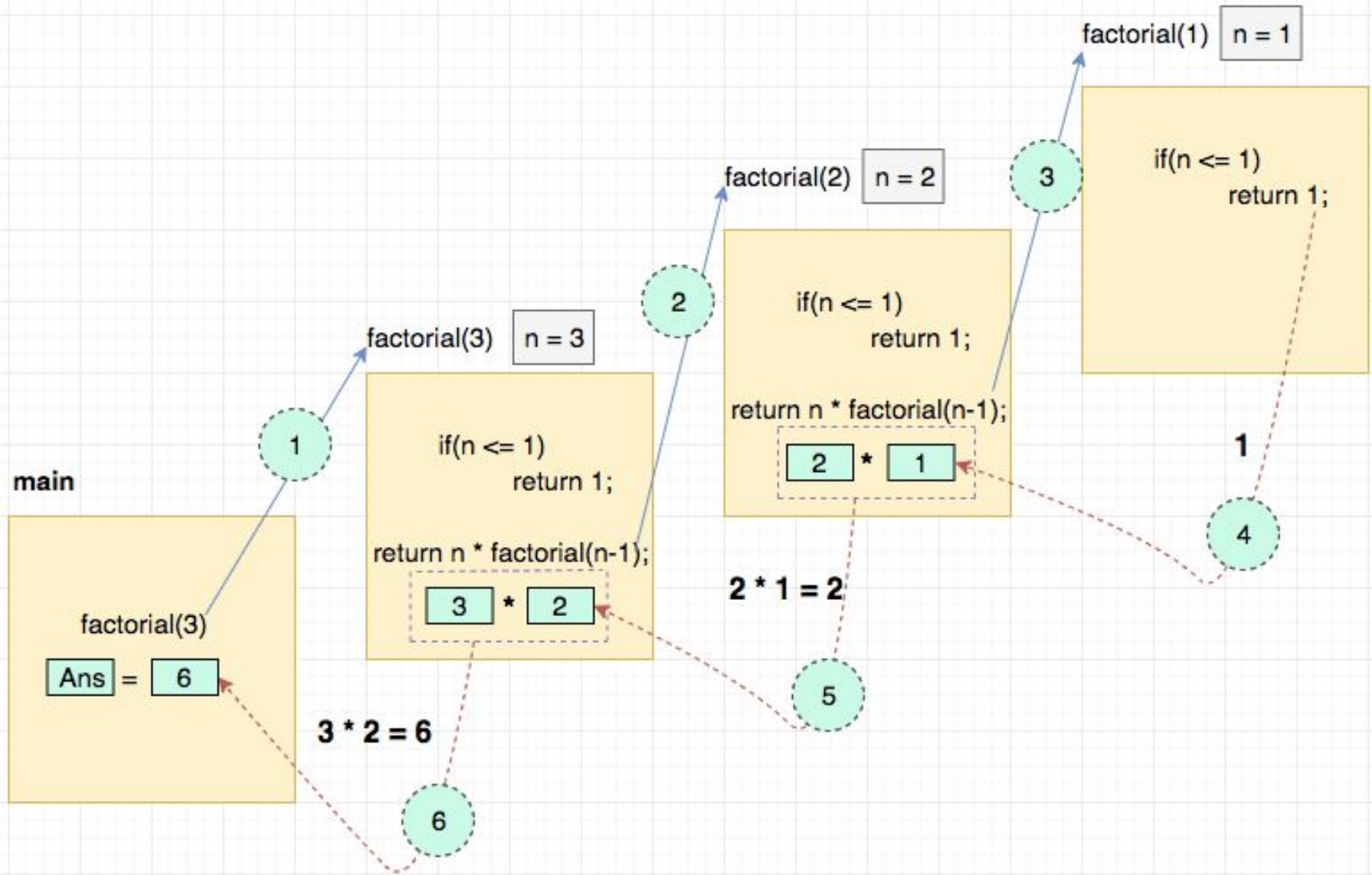
Recursión

```
fact(5)
= 5 * fact(4)
= 5 * 4 * fact(3)
= 5 * 4 * 3 * fact(2)
= 5 * 4 * 3 * 2 * fact(1)
= 5 * 4 * 3 * 2 * 1 * fact(0)
```

Recursión



Recursión



Ventajas y Desventajas de la recursión

Diseño simple y elegante.

Soluciones más fáciles de entender y leer.

Pero...

Ocupa demasiada memoria con cada auto invocación.

Demanda mayor tiempo de ejecución comparado con la misma solución iterativa.

Ejercicio 2

- Definir una función recursiva que reciba un número entero y devuelva la suma desde 1 hasta dicho número.



Ejercicio 2

- Definir una función recursiva que reciba un número entero y devuelva la suma desde 1 hasta dicho número.

```
def sumar(n):  
    return 1+2+3+....+n
```

```
def sumar (n):
```

```
    if (n == 1):
```

```
        return n
```

```
    else:
```

```
        return n + sumar(n-1)
```

Ejercicio 3

- Definir una función recursiva que reciba una lista de apellidos y los imprima.



Ejercicio 3 - solución 1

- Definir una función recursiva que reciba una lista de apellidos y los imprima.

```
def imprimir_lista(lista, posicion):  
    if (posicion != -1):  
        print(lista[posicion])  
        imprimir_lista(lista, posicion - 1)
```

```
lista = ["perez", "sand", "arlt", "claymore"]  
imprimir_lista(lista, len(lista) - 1)
```

Ejercicio 3 - solución 2

- Definir una función recursiva que reciba una lista de apellidos y los imprima.

```
def imprimir_lista(lista, posicion):  
    if (posicion < len(lista)):  
        print(lista[posicion])  
        imprimir_lista(lista, posicion + 1)
```

```
lista = ["perez", "sand", "arlt", "claymore"]  
imprimir_lista(lista, 0)
```

